CMO DYNAMIC CAMPAIGN

CURRENT CODE CAPABILITY GUIDE

What the framework actually handles today - and where its current boundary lies

MEETING EDITION / 7 SEPTEMBER 2026

Public repository: murdockpeter/cmo_dynamic_campaign

Prepared as a discussion guide for fellow wargamers and collaborators.

336	299	27	10	10
TRACKED MOBILE ELEMENTS	AIRCRAFT	SURFACE SHIPS	SUBMARINES	MISSIONS

CORE PREMISE

Use C:MO as the physical combat-resolution engine while keeping stable campaign identity, reviewable scenario construction, end-of-day evidence, adjudication, and future operational theory outside the opaque scenario payload.

01 Executive briefing

The shortest version to present verbally before diving into the architecture.

CURRENT	The repository already creates a deterministic, playable South China Sea Day 1 scenario through supported C:MO Lua APIs, with stable campaign identities, preflight navigation safety, post-build verification, an end-of-day finalizer, and a separate operational picture application.

BOUNDARY	The campaign ledger, generalized next-day transition engine, detailed logistics accounting, modular theory SDK, and Entropy model are design targets - not current runtime capabilities.

What is working today

- A Python generator acts as the executable Day 1 specification and expands force packages into individually tracked aircraft, ships, and submarines.
- C:MO itself constructs and serializes the playable scenario. The project deliberately does not reverse-engineer or directly write the proprietary compressed scenario body.
- Every authored mobile element receives a stable campaign ID and a deterministic UUIDv5 planned C:MO GUID.
- Seven complex airbase/island installation templates are imported instead of rebuilding hundreds of runway, parking, magazine, sensor, and support objects by hand.
- Surface and submarine starts/routes are fail-closed through an offline coastline/hazard check, automatic route repair, and a second C:MO terrain/bathymetry gate.
- The built scenario contains ten preplanned CAP, AEW, tanker, and ASW missions plus an escalation-restriction area.
- A post-build Lua validator checks all 336 mobile elements for presence, expected GUID, expected DBID, and verifies all sixteen assigned navigation routes against C:MO terrain/depth.
- A BLUE special action finalizes Day 1 by printing losses/expenditures and survivor summaries, exporting each side, and saving day-001-final.save.
- A desktop Electron/Google Maps tracker renders the current planning baseline using manifest.json and input.json, with filters, mission geometry, routes, restrictions, and element drill-down.

What not to overclaim in the meeting

Area	Current truth
Persistent multi-day ledger	Documented architecture, but not yet implemented as campaign/ledger/transaction files.
Automated Day 2 transition	Documented target-state workflow; no current generalized transition-day-002 runtime pipeline.
Detailed logistics	CMO exposes useful physical state, but no full stockpile/transport/replenishment accounting module exists yet.
Entropy / theory plug-ins	Proposed in the modular coding plan; not part of current runtime code.
Six-hour checkpoints	Described in workflow docs, but the current build does not create timed checkpoint events.
Deep final report	Current finalizer is narrower: losses, expenditures, GUID/name/DBID, position, damage %, fuel state, weapon state, side exports, final save.

02 Architecture and trust boundaries

The strongest architectural idea already present is the separation between authored campaign state and engine-owned tactical state.

End-to-end pipeline

Stage	Owner	What happens
1. Author	Workspace / Python	Force packages, templates, loadouts, starts, routes, missions, campaign rules, and deterministic identity are defined.
2. Generate	Workspace / Python	Generator emits input.json, manifest.json, route audits, preflight.lua, build.lua, validate.lua, and RUNME.txt.
3. Preflight	C:MO Lua	A blank DB3K_515 scenario is created and all 16 routes are sampled against C:MO terrain/bathymetry.
4. Construct	C:MO Editor	Templates import; 336 mobile elements are added; courses, missions, side settings, and key/value flags are created.
5. Validate	C:MO Lua	All expected mobile identities and actual assigned routes are checked.
6. Serialize / Play	C:MO	C:MO writes the opaque .scen/.save body and resolves the physical game-day.
7. Finalize	C:MO + workflow	Losses/expenditures and survivor summaries are emitted, survivor sets are exported, and a final save is written.

Two authorities

CMO	Authoritative for what physically happened during the played game-day: movement, combat, damage, fuel use, weapon expenditure, surviving units, and other engine state.

CAMPAIGN	Intended authority between game-days: stable identity, human adjudication, repair/replacement/replenishment decisions, historical record, and future theory modules.

Identity strategy

Every persistent mobile element has two identities: a human-readable stable campaign ID and a C:MO GUID/DBID locator. The generator computes planned GUIDs using UUIDv5 from a fixed namespace, so regenerating the same campaign ID produces the same intended GUID.

Example stable ID	Meaning
BLU-USN-CVN-0001	Individual BLUE carrier
BLU-USAF-F35A-0007	Individual USAF F-35A tail-equivalent
RED-PLAN-SSN-0002	Individual PLAN SSN
RED-PLANAF-J15-04	Individual carrier aircraft

Important current source-of-truth wrinkle

PARTIAL	Although design docs describe input.json as the frozen pre-generation input, the current generator rebuilds input.json from Python literals. Today, tools/generate_day1.py is the actual authoritative Day 1 specification.

03 What the Day 1 generator actually builds

A concrete South China Sea crisis baseline, generated deterministically into a C:MO editor script.

165	171	7	6	16
BLUE MOBILE	RED MOBILE	SITE TEMPLATES	SURFACE GROUPS	NAV ROUTES

Scenario envelope

Field	Current value / behavior
Campaign	SCS-2026
Scenario	Fault Line - South China Sea Day 1
Database	DB3K_515.db3
Start	15 Aug 2026 0000Z
Operational pause	16 Aug 2026 0000Z
Sides	BLUE vs RED, mutually Hostile
Player	BLUE; RED is computer-controlled-only
Proficiency	Regular for both sides / generated mobile units
Escalation flags	Mainland strikes false; Guam strikes false

Mobile force accounting

Side	Aircraft	Surface	Submarines	Total
BLUE	150	11	4	165
RED	149	16	6	171
TOTAL	299	27	10	336

Land-based aviation represented

The generator expands package counts into individually named aircraft, with exact DBIDs, assigned bases, planned GUIDs, and ready/unready loadout behavior.

BLUE examples	RED examples
FA-50PH, F-16C Block 50, F-35B	J-16, H-6J, J-10, J-20
P-8A, MQ-9A, KC-130J	Y-8Q, KJ-500/KJ-500H, GJ-2
F-35A, KC-135R, E-3, RC-135	J-16D, YY-20, Y-9JB
C-130J, HH-60W	Woody Island / Fiery Cross detachments

Embarked aviation represented

BLUE carrier / ARG aviation	RED carrier aviation
F-35C, F/A-18E, F/A-18F, EA-18G	J-15, J-15D
E-2D, MH-60R, MH-60S, CMV-22	Z-18J, Z-18F, Z-9D
F-35B, MV-22, CH-53K, AH-1Z, UH-1Y	Shandong-based packages

Complex installation reuse

- BLUE: Basa Air Base; Antonio Bautista / Puerto Princesa; Mactan-Benito Ebuen.
- RED: Lingshui; Suixi; Woody Island; Fiery Cross Reef.

04 Naval groups, submarines, and movement

Ships are individually identifiable while still operating in authored task groups and routes.

Six surface task groups

Task group	Side	Composition
TG B-1 George Washington CSG	BLUE	CVN + 3 DDGs + AOE
TG B-2 Philippine West Sea SAG	BLUE	2 Philippine frigates + corvette
TG B-3 America ARG	BLUE	LHA + LPD + DDG
TG R-1 Shandong CSG	RED	CV + 3 DDGs + FFG + AOE
TG R-2 Southern Theater SAG	RED	3 DDGs + FFG + AOR
TG R-3 Hainan Amphibious Group	RED	LHD + 2 LPDs + DDG + FFG

Ten individually tracked submarines

Side	Count	Current generated categories
BLUE	4	Four individually identified SSNs with assigned patrol courses.
RED	6	Two SSNs, three SSKs, and one SSBN, each with a generated course.

Movement semantics

- Ships are created individually around a task-group center using small fixed positional offsets, then the group receives an authored/resolved course.
- Submarines are created individually and receive a course plus manual depth of approximately -150 m in the generated build.
- The offline route system enforces 3 nm minimum horizontal clearance for surface groups and 2 nm for submarines.
- C:MO preflight additionally requires submarine route samples to have at least 200 m of water depth.
- Player-, AI-, mission-, or event-generated course changes after build time are outside the current navigation guarantee.

DESIGN CHOICE	Task-group membership is operational grouping, not identity. Individual ships retain their own campaign IDs so they can survive group changes across future campaign days.

05 Mission and operational tasking

The current build creates ten mission objects with reference-point geometry and assigns selected aircraft.

Side	Mission	Type	Assigned force idea
BLUE	West Luzon CAP	AAW Patrol	F-16s plus FA-50s
BLUE	Palawan CAP	AAW Patrol	F-35Bs
BLUE	AEW Central	Support	E-3
BLUE	Tanker Central	Support	KC-135s + KC-130
BLUE	Palawan ASW	ASW Patrol	P-8As
RED	Hainan CAP	AAW Patrol	J-16/J-10/J-20 mix
RED	Spratly CAP	AAW Patrol	Fiery Cross J-10s
RED	AEW Hainan	Support	KJ-500 variants
RED	Tanker Hainan	Support	YY-20s
RED	Central Basin ASW	ASW Patrol	Y-8Q aircraft

What the mission builder sets

- Patrol or Support mission type.
- Reference-point mission geometry.
- Active state.
- On-station count.
- One-third rule and patrol checks for OPA/WWR where applicable.
- Loop behavior for support missions.
- Explicit unit-to-mission assignment by generated GUID.

Escalation geometry

A mainland strike restriction polygon is carried in the generated planning data. The scenario also stores key/value flags for mainland and Guam strike authorization. This is an important foundation for later political/escalation modules, but the current build does not yet implement a rich trigger tree that automatically changes those permissions.

Current validation limit

NOT VERIFIED	Post-build validation proves unit identity and navigation, but it does not currently verify that all ten missions exist with the correct zones, assignments, or semantic settings.

06 Navigation safety - the most mature guardrail

Route generation is intentionally fail-closed and uses both external geodata and C:MO's own terrain model.

Layer 1 - offline geometry gate

- Checks all 27 surface-ship starts and all 10 submarine starts.
- Checks all six surface-group routes and all ten submarine routes before build.lua is written.
- Uses a checked-in Natural Earth 1:10m South China Sea land/minor-island crop.
- Adds ten conservative supplemental hazard buffers around selected reefs, shoals, and artificial-island features.
- Performs exact segment/coast intersection tests.
- Densifies great-circle routes and samples distance-to-land at 0.25 nm intervals.
- Relocates invalid intermediate waypoints to safe water, inserts detours for unsafe legs, simplifies where possible, then re-runs the full validator.
- Stops generation if a safe repair cannot be found.

Layer 2 - C:MO terrain and bathymetry corroboration

- preflight.lua creates a fresh blank scenario using DB3K_515.
- Every emitted route is sampled at 0.5 nm intervals using World_GetElevation().
- Great-circle sample points are advanced using Tool_Bearing() and World_GetPointFromBearing().
- Surface route sample >= 0 m elevation fails.
- Submarine route sample on land or shallower than 200 m fails.
- Terrain lookup failure is fatal.
- Only a clean pass writes dc.navigation.preflight=true; build.lua refuses to run without that key.

Layer 3 - post-build assigned-course validation

validate.lua re-reads each actual C:MO group/submarine course, checks that at least the expected number of waypoints exists, and re-samples the assigned course against C:MO terrain/depth.

GUARANTEE	The initial starts and emitted/assigned courses are guarded at generation time. The system does not police routes later changed by the player, mission AI, Lua events, or other runtime behavior.

07 Validation and automated tests

The project validates some things rigorously and intentionally does not pretend that this equals full scenario correctness.

What validate.lua proves

Surface	Status	Current proof
336 expected mobile names	Implemented	Every expected unit is resolved on its expected side.
GUID	Implemented	Actual C:MO GUID is compared with deterministic planned GUID.
DBID	Implemented	Actual platform DBID is compared with expected DBID.
16 assigned courses	Implemented	Waypoint count and sampled terrain/depth are checked.
Loadout / host / readiness	Not checked	Aircraft can exist while being based or configured incorrectly.
Mission semantics	Not checked	Missions/assignments/zones are not post-build verified.
Template composition	Partial	Import must return >0, but member-by-member expected composition is not compared.
Saved-file equivalence	Not checked	No current hash ties validated in-memory state to the later saved artifact.

Python navigation test suite

- Point-on-land detection.
- Exact route intersection handling.
- Multi-waypoint repair.
- Invalid-waypoint relocation.
- Supplemental Scarborough Shoal detection.
- Regression coverage for Day 1 routes that have required automatic repair.

STRONG	Unsafe starts, unresolved route repairs, C:MO land hits, shallow submarine paths, or missing preflight completion stop scenario construction instead of issuing a warning and continuing.

08 Saving, finalization, and campaign evidence

The code already preserves a useful boundary between a reproducible starting scenario and a physical end-of-day record.

Three save moments

Moment	Mechanism	Meaning
Build completion	Command_SaveScen()	C:MO serializes the just-built Day1.scen baseline.
Human Save As	Scenario Editor UI	Operator-reviewed starting scenario.
End-of-day finalization	BLUE special action	Writes day-001-final.save after the played day and emits side/export evidence.

What the current finalizer does

1. Checks dc.day001.finalized; exits if already marked.
2. Sets the finalized marker.
3. For BLUE and RED, prints side loss records.
4. Prints side expenditure records.
5. Enumerates surviving side units.
6. For each survivor, prints GUID, name, DBID, latitude, longitude, damage percentage, fuel state, and weapon state.
7. Exports each side's current survivor set to a C:MO .inst package.
8. Writes day-001-final.save.
9. Prints DCREPORT|FINALIZED|001|<time> and displays a completion message.

Current finalizer caveats

ATOMICITY	The finalized marker is set before reporting/export/save. If a later operation fails, the scenario can be marked finalized even though the output package is incomplete.
SCOPE	The side export includes every surviving unit returned by the side, including imported site/template members - not just the 336 campaign-manifest mobile elements.
REPORT DEPTH	The current report is intentionally smaller than the design target. It does not yet print detailed base/group/mission/loadout/mount/magazine records or automated 6/12/18-hour checkpoints.

Why the .save chain matters

The target campaign workflow continues from C:MO save to C:MO save rather than rebuilding the entire theater every day. This is designed to preserve difficult-to-reconstruct tactical detail such as position, damage, hosted aircraft, contacts, fuel, magazines, mission state, and engine knowledge. The external campaign layer is then intended to apply only approved administrative changes between days.

09 Situation tracker / common operational picture

A separate desktop planning application visualizes the frozen Day 1 baseline; it is not live C:MO telemetry.

7	6	10	10	336
AIRBASE/ISLAND NODES	NAVAL GROUPS	SUBMARINES	MISSION LAYERS	DRILL-DOWN IDS

Current tracker behavior

- Electron desktop application using Google Maps JavaScript API.
- Reads days/day-001/manifest.json and input.json at runtime; route and mission geometry are not duplicated in the current situation adapter.
- Aggregates aircraft into base/island or ship-group nodes and surfaces submarines individually.
- Derives a simple ready/armed count from whether aircraft loadout_id is 4.
- Displays force markers, surface/submarine routes, mission polygons/lines, and the mainland escalation restriction.
- Provides side filters, text search, route/mission/restriction layer toggles, terrain/hybrid basemap, node drill-down, platform DBIDs, loadout IDs, and stable campaign IDs.
- Watches manifest.json and input.json for changes and reloads the data view.

Local-app behavior and security

- Binds its static server only to 127.0.0.1:43117.
- Blocks path traversal and external navigation/window creation.
- Uses Node integration off, context isolation on, and sandboxing on.
- Stores newly entered Maps keys with Electron safeStorage when OS-backed encryption is available.

What the tracker is not

NOT LIVE	Positions are the frozen generated Day 1 planning baseline. The tracker does not read a running C:MO session and does not function as an authoritative tactical common operating picture.

NOT VALIDATOR	Google Maps is visual context only. Route safety authority remains route-audit plus C:MO preflight/validate sampling.

10 CMO database and template catalog tooling

A read-only inspection utility lets scenario generation be grounded in the installed C:MO database instead of guessed identifiers.

cmo_catalog.py

- Locates the installed C:MO root from a default Steam path, explicit argument, or CMO_ROOT environment variable.
- Enumerates installed .db3 files and identifies the latest database in each family.
- Opens SQLite database files in URI mode=ro so the utility cannot write them.
- Searches aircraft, ships, submarines, facilities, ground units, weapons, and loadouts by name.
- Supports optional in-service year filtering when the table exposes commissioning/decommissioning fields.
- Can explicitly pin a database revision instead of using the newest DB3K.
- Parses JSON .inst templates and summarizes member count and geographic center.

Why database pinning matters

A DBID is only meaningful in the context of a database revision. The current Day 1 build pins DB3K_515.db3, and the repository treats installed database files as read-only reference data.

Installation reconnaissance that shaped the design

Observed capability	Implication for the framework
CMO Lua can build a blank scenario	Supported Lua/editor path can be used instead of manipulating compressed scenario internals.
.inst templates preserve complex installations	Airfields and site systems can be imported as detailed reusable physical structures.
Lua exposes units, losses, expenditures, damage/fuel/magazines/mounts/etc.	Enough raw state exists to support richer future reconciliation and logistics modules.
CMO .campaign format is simple/linear	Desired operational persistence must live in the external framework rather than relying on built-in campaign XML alone.

11 Generated artifacts and operator workflow

The framework is evidence-preserving: generated text can be inspected and engine-owned outputs can be retained separately.

Day 1 package

Artifact	Current role
input.json	Generated scenario envelope, navigation geometry, missions, boundaries, campaign metadata.
manifest.json	336 stable campaign IDs mapped to side, kind, DBID, planned GUID, and platform-specific fields.
preflight.lua	C:MO-native initial terrain/bathymetry gate.
build.lua	Full generated scenario construction program.
validate.lua	Post-build mobile identity and assigned-course check.
route-audit.json	Machine-readable navigation audit and provenance.
route-audit.html	Human-readable route audit map/report.
RUNME.txt	Operator handoff instructions for preflight, build, validate, and Save As.
Day1.scen	Playable C:MO baseline serialized by the engine.
day-001-final.save	End-of-game-day authoritative physical checkpoint when finalization is executed.

Current operator sequence

1. Review route-audit.html and confirm the generation audit is clean.
2. Open C:MO in Scenario Editor mode.
3. Run preflight.lua from the Lua Script Console.
4. Wait for the preflight completion marker.
5. Run build.lua; the build refuses to proceed without the preflight key.
6. Run validate.lua and review the DCVALIDATE summary/history.
7. Inspect bases, aircraft hosts/loadouts, missions, groups, and courses manually where semantic validation does not yet exist.
8. Use Editor > Save As Scenario to preserve the reviewed Day1.scen.
9. Play the agreed game-day.
10. At the 24-hour pause, invoke Finalize Game-Day 1 once and verify the final save, side exports, and Lua-history completion record.

12 Current implementation boundary

A clean separation between implemented now and designed next will make the meeting much more productive.

Capability	Status	Notes
Deterministic Day 1 generator	IMPLEMENTED	Python expands authored force packages and writes reviewable Lua/artifacts.
Stable campaign IDs / planned GUIDs	IMPLEMENTED	UUIDv5 strategy used for 336 mobile elements.
Pinned CMO database	IMPLEMENTED	DB3K_515.
Complex site import	IMPLEMENTED	Seven .inst templates.
Surface/sub navigation gate + repair	IMPLEMENTED	Offline + C:MO preflight + post-build check.
Preplanned CAP/AEW/tanker/ASW missions	IMPLEMENTED	Ten missions.
Post-build identity validation	IMPLEMENTED	Name/side/GUID/DBID.
End-of-day final save/export/report	IMPLEMENTED	Narrow finalizer.
Desktop COP / tracker	IMPLEMENTED	Frozen planning baseline, not live telemetry.
Persistent ledger + transactions	DESIGNED	Documented but not present as an active campaign engine.
Automated Day N -> Day N+1 transitions	DESIGNED	Workflow exists on paper; no general transition engine yet.
Detailed fuel/munition/spares logistics	NEXT	Natural major module target.
Maintenance/repair scheduling	NEXT	Not currently modeled by external rules engine.
Theory module SDK	PROPOSED	BYOT architecture plan exists but is not runtime code.
Entropy module	PROPOSED	Conceptual reference module; not implemented.
Rich checkpoint/event system	PARTIAL	Key/value flags and final special action exist; timed 6/12/18h checkpoints do not.

Most important present-day technical gaps

- Finalizer atomicity: mark-finalized occurs before export/save completion.
- input.json is generated output rather than the true declarative source of the generator.
- Loadout, host, readiness, mission, and template semantic correctness are not comprehensively post-build validated.
- No saved-artifact hash/build receipt ties the validated in-memory scenario to the exact saved .scen file.
- The external ledger / transaction history / automated reconciliation pipeline remains target architecture rather than working code.
- Some documentation deliberately describes the future end-state and is ahead of current implementation.

13 How to explain the wargaming value

The project is best framed not as automation for C:MO, but as a separation between tactical resolution and operational persistence.

Three experimental layers already visible

Layer	Current role	Wargaming value
Authored campaign layer	Inspectable Python/JSON definitions, stable IDs, mission geometry, restrictions	Makes assumptions visible and reproducible instead of burying them inside a scenario file.
C:MO tactical layer	Sensors, weapons, movement, damage, expenditure, AI, geography	Lets a mature simulation resolve physical interaction without the campaign framework reinventing combat mechanics.
Evidence layer	Save files, side exports, Lua-history records, manifests, audits	Creates a trail that can support adjudication, AAR, and eventually controlled theory modules.

What this makes possible next

- Detailed logistics and operational reach.
- Maintenance and sortie-generation constraints.
- Herman-inspired entropy/cohesion.
- Command and information uncertainty.
- Network fragility and C2 degradation.
- Operational tempo / decision-cycle experiments.
- Political or escalation rules.

DISCUSSION POINT	The strongest future design principle is that optional theory modules should observe common campaign facts and propose bounded effects - not directly rewrite C:MO truth or one another's private state.

Questions worth asking your fellow wargamer

1. Which operational phenomena are most valuable to model outside C:MO rather than inside the scenario itself?
2. Where should player judgment remain human adjudication instead of becoming a deterministic rule?
3. What logistics granularity creates meaningful command decisions without turning the campaign into warehouse bookkeeping?
4. Which state should be considered objective physical truth, and which state should be theory-dependent interpretation?
5. What would make a plug-in theory module scientifically or professionally credible: sources, assumptions, calibration, traceability, A/B comparison?
6. Which current validation gaps matter most before moving into a multi-day campaign?

A Repository source map used for this guide

Grounded in the public repository current main branch as reviewed on 5 September 2026.

Source	What it establishes
README.md	Overall campaign loop, guardrails, tracker summary, current navigation-safety claim.
tools/generate_day1.py	Actual authoritative Day 1 OOB/packages, templates, missions, GUID generation, route validation/repair, emitted Lua, and deployment.
tools/navigation.py	Offline coastline/hazard geometry, clearance calculations, route repair and simplification.
tools/cmo_catalog.py	Read-only installed-database and .inst discovery/search behavior.
days/day-001/input.json	Current campaign envelope, resolved navigation routes, mission/boundary data consumed by the tracker.
days/day-001/manifest.json	Stable mobile-element identity map.
days/day-001/preflight.lua	C:MO-native route terrain/depth gate.
days/day-001/build.lua	Actual generated C:MO scenario construction and current finalizer.
days/day-001/validate.lua	Actual post-build identity and route validation.
days/day-001/route-audit.json	Current route-audit settings, hazards, spawn checks, and route results.
tracker/src/*.cjs + renderer/app.js	Current desktop COP data flow, security settings, filtering, map layers, and runtime behavior.
docs/campaign-model.md	Target two-layer persistence model and reconciliation principles.
docs/daily-workflow.md	Target multi-day save-chain and human/Codex handoff; some items are ahead of implementation.
docs/navigation-safety.md	Implemented navigation safety architecture and explicit limits.
docs/reconnaissance.md	Initial C:MO installation/API/database reconnaissance and design rationale.
docs/CMO_DYNAMIC_CAMPAIGN_MODULAR_THEORY_CODING_PLAN.md	Future module SDK / Bring Your Own Theory architecture; not current runtime capability.

Public repository

https://github.com/murdockpeter/cmo_dynamic_campaign

One-sentence close

BOTTOM LINE	Today the repo can deterministically construct, safety-check, validate, visualize, play, and finalize a substantial 336-element C:MO Day 1 baseline. The next architectural leap is turning that strong single-day evidence pipeline into a general multi-day campaign kernel with auditable plug-in theories and deep logistics.